REST API and Persistent Data - Source Code

■■ PDF ■■■■ Node.js, Express ■■ SQLite ■■■■■■■■■■ ■■ ■■■■ ■■■■ ■■■■■■■ ■■ ■■ ■■■■ ■■■■■■■■■ ■■■■■ ■■■■

1. package.json

```
{
  "name": "internship-api",
  "version": "1.0.0",
  "description": "REST API for Internship Records",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "sqlite3": "^5.1.7"
  }
}
```

2. database.js

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database(':memory:'); // In-memory database for testing

db.serialize(() => {
  // 1. Create Data Model Schema
  db.run(`
    CREATE TABLE internships (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      candidateName TEXT NOT NULL,
      role TEXT NOT NULL,
      company TEXT NOT NULL,
      status TEXT NOT NULL,
      duration TEXT
    )
  `);

  // 2. Insert Seed Data
  const insert = db.prepare(`
    INSERT INTO internships (candidateName, role, company, status, duration)
    VALUES (?, ?, ?, ?, ?)
  `);

  insert.run('Amit Sharma', 'Backend Developer Intern', 'TechCorp', 'Accepted', '6 Months');
  insert.run('Neha Gupta', 'Frontend Developer Intern', 'WebSystems', 'Interviewing', '3 Months');
  insert.run('Rahul Verma', 'Data Science Intern', 'DataAI', 'Applied', '4 Months');

  insert.finalize();
  console.log('Database initialized with seed data.');
});

module.exports = db;
```

3. server.js

```
const express = require('express');
const db = require('./database');
const app = express();
const PORT = 3000;

app.use(express.json());

// Helper function for input validation
const validateInternship = (data) => {
  const { candidateName, role, company, status } = data;
  if (!candidateName || !role || !company || !status) {
    return 'candidateName, role, company, and status are required fields.';
  }
  return null;
};

// 1. LIST EndPoint (With Pagination)
app.get('/api/internships', (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 2;
  const offset = (page - 1) * limit;

  db.all('SELECT * FROM internships LIMIT ? OFFSET ?', [limit, offset], (err, rows) => {
    if (err) return res.status(500).json({ error: err.message });
    res.json({ page, limit, data: rows });
  });
});

// 2. DETAIL EndPoint
app.get('/api/internships/:id', (req, res) => {
  db.get('SELECT * FROM internships WHERE id = ?', [req.params.id], (err, row) => {
    if (err) return res.status(500).json({ error: err.message });
    if (!row) return res.status(404).json({ error: 'Record not found' });
    res.json(row);
  });
});

// 3. CREATE EndPoint (With Validation)
app.post('/api/internships', (req, res) => {
  const error = validateInternship(req.body);
  if (error) return res.status(400).json({ error });

  const { candidateName, role, company, status, duration } = req.body;
  const sql = `INSERT INTO internships (candidateName, role, company, status, duration) VALUES (?,

  db.run(sql, [candidateName, role, company, status, duration], function (err) {
    if (err) return res.status(500).json({ error: err.message });
    res.status(201).json({ id: this.lastID, ...req.body });
  });
});

// 4. UPDATE EndPoint (With Validation)
app.put('/api/internships/:id', (req, res) => {
  const error = validateInternship(req.body);
  if (error) return res.status(400).json({ error });
```

```
  const { candidateName, role, company, status, duration } = req.body;
  const sql = `UPDATE internships SET candidateName = ?, role = ?, company = ?, status = ?, durati

  db.run(sql, [candidateName, role, company, status, duration, req.params.id], function (err) {
    if (err) return res.status(500).json({ error: err.message });
    if (this.changes === 0) return res.status(404).json({ error: 'Record not found' });
    res.json({ message: 'Record updated successfully', id: req.params.id, ...req.body });
  });
});

// 5. DELETE EndPoint
app.delete('/api/internships/:id', (req, res) => {
  db.run('DELETE FROM internships WHERE id = ?', [req.params.id], function (err) {
    if (err) return res.status(500).json({ error: err.message });
    if (this.changes === 0) return res.status(404).json({ error: 'Record not found' });
    res.json({ message: 'Record deleted successfully' });
  });
});

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

4. README.md

```
# Internship Records REST API

A simple Node.js and Express API to manage internship records with SQLite persistence, validation,

## Setup Instructions
1. Clone this repository.
2. Install dependencies:
   npm install
3. Start the server:
   npm start

## API Examples
- GET /api/internships?page=1&limit=2 (Get All Internships with Pagination)
- POST /api/internships (Create an Internship)
```